

Question 3:

code:

```
// Code for modeling a university's various units: department, office,
// college, etc

// Generic interface.
#include <vector>

class EUnit
{
public:
    virtual ~EUnit() {}
    virtual double GetBudget() const = 0;
};

// Department
class ECDepartment : public EUnit
{
public:
    ECDepartment(int b) : budget(b) {}

    double GetBudget() const override
    {
        return budget;
    }

private:
    int budget;
};

// Office
class ECOffice : public EUnit
{
public:
    ECOffice(int b) : budget(b) {}

    double GetBudget() const override
    {
        return budget;
    }

private:
    int budget;
};

// Composite
class ECCompositeUnit : public EUnit
{
public:
    ECCompositeUnit() {}

    ~ECCompositeUnit()
    {
        for (EUnit* p : children)
        {
            delete p;
        }
    }
}
```

```

void AddChild(ECUnit *pUnit)
{
    children.push_back(pUnit);
}

double GetBudget() const override
{
    double sum = 0;
    for (ECUnit* p : children)
    {
        sum += p->GetBudget();
    }
    return sum;
}

private:
    std::vector<ECUnit*> children;
};

```

output:

```

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ clang++ EOrganizationTest.1.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ ./a.exe
School budget: 310

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ clang++ EOrganizationTest.2.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ ./a.exe
School of Engineering budget: 400
University budget: 720

```

Question 4:

code:

.cpp:

```
#include "ECSportsPlayer.h"

// *****
// Generic player
bool ECSportsPlayer::CanPlay(ECTournament &tr) const
{
    return false;
}

// *****
// Ranked player
ECSportsPlayerRanked::ECSportsPlayerRanked(int r) : rank(r)
{
}

bool ECSportsPlayerRanked::CanPlay(ECTournament &tr) const
{
    return rank <= tr.GetMinRank();
}

// *****
// Wildcard player: qualify for any tournament
ECSportsPlayerWildcard::ECSportsPlayerWildcard(ECSportsPlayer &playerOrig) :
    ECSportsPlayerDecorator(playerOrig)
{
}

bool ECSportsPlayerWildcard::CanPlay(ECTournament &tr) const
{
    return true;
}

// *****
// Disciplined player: cannot attend any tournament during the period of probation
ECSportsPlayerDisciplined::ECSportsPlayerDisciplined(ECSportsPlayer &po, int mStart, int mEnd)
    : ECSportsPlayerDecorator(po), ms(mStart), me(mEnd)
{
}

bool ECSportsPlayerDisciplined::CanPlay(ECTournament &tr) const
{
    int tm = tr.GetMonth();

    if (tm >= ms && tm <= me)
        return false;

    return player.CanPlay(tr);
}

// *****
// Injured player: cannot attend any tournament during the period of injury(injury occurs at a
single month)
```

```

ECSportsPlayerInjured::ECSportsPlayerInjured(ECSportsPlayer &po, int m) :
ECSportsPlayerDecorator(po), mInj(m)
{
}

bool ECSportsPlayerInjured::CanPlay(ECTournament &tr) const
{
    if (tr.GetMonth() == mInj)
        return false;

    return player.CanPlay(tr);
}

```

.h:

```

#ifndef _EC_SPORTS_PLAYER_H
#define _EC_SPORTS_PLAYER_H

// *****
// Sports Tournament
class ECTournament
{
public:
    ECTournament(int mon, int mr) : month(mon), minRank(mr) {}
    int GetMonth() const { return month; }
    int GetMinRank() const { return minRank; }

private:
    int month;
    int minRank;
};

// *****
// Generic player
class ECSportsPlayer
{
public:
    virtual ~ECSportsPlayer() {}
    virtual bool CanPlay(ECTournament &tr) const = 0;
};

// *****
// Ranked player
class ECSportsPlayerRanked : public ECSportsPlayer
{
public:
    ECSportsPlayerRanked(int r);
    bool CanPlay(ECTournament &tr) const override;

private:
    int rank;
};

// *****
// Decorator base class
class ECSportsPlayerDecorator : public ECSportsPlayer
{
public:
    ECSportsPlayerDecorator(ECSportsPlayer &p) : player(p) {}

```

```

protected:
    ECSportsPlayer &player;
};

// *****
// Wildcard player: qualify for any tournament
class ECSportsPlayerWildcard : public ECSportsPlayerDecorator
{
public:
    ECSportsPlayerWildcard(ECSportsPlayer &playerOrig);
    bool CanPlay(ECTournament &tr) const override;
};

// *****
// Displined player: cannot attend any tournament during the period of probation
class ECSportsPlayerDisciplined : public ECSportsPlayerDecorator
{
public:
    ECSportsPlayerDisciplined(ECSportsPlayer &playerOrig, int mStart, int mEnd);
    bool CanPlay(ECTournament &tr) const override;

private:
    int ms, me;
};

// *****
// Injured player: cannot attend any tournament during the period of injury(injury occurs at a
single month)
class ECSportsPlayerInjured : public ECSportsPlayerDecorator
{
public:
    ECSportsPlayerInjured(ECSportsPlayer &playerOrig, int m);
    bool CanPlay(ECTournament &tr) const override;

private:
    int mInj;
};

#endif

```

output:

```

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ clang++ ECSportsPlayerTest.cpp ECSportsPlayer.cpp

karin@Karinas_Laptop CLANG64 /c/Users/karin/Music/CSE-3150/lab10
$ ./a.exe
Player 1: NOT qualified
Player 2: qualified
Player 3: NOT qualified
Player 4: NOT qualified
Player 5: qualified
Player 6: qualified
Player 7: qualified

```